Learning to Accelerate Vision-Language-Action Models through Adaptive Visual Token Caching

Yujie Wei[1*] Jiahan Fan[2*] Jiyu Guo[2] Ruichen Zhen[3] Rui Shao[2]
Xiu Su[4] Zeke Xie[5] Hongxun Yao[1] Shuo Yang[2✉]

[1]Harbin Institute of Technology [2]Harbin Institute of Technology, Shenzhen
[3]Meituan Academy of Robotics Shenzhen, Meituan [4]Central South University [5]HKUST(GZ)

Abstract

Vision-Language-Action (VLA) models have demonstrated remarkable generalization capabilities in robotic manipulation tasks, yet their substantial computational overhead remains a critical obstacle to real-world deployment. Improving inference efficiency is therefore essential for practical robotic applications. Existing acceleration methods often rely on heuristic or static strategies—such as rule-based token caching or pruning—that are decoupled from task objectives and fail to adapt to dynamic scene changes. In this work, we reformulate inference acceleration as a learnable policy optimization problem and propose a novel framework that integrates a dynamic, task-aware decision-making process directly into the VLA model. At its core are two lightweight, cooperative modules: a Cached Token Selector, which determines which tokens should be reused, and a Cache Ratio Predictor, which controls how many tokens to reuse. Training these modules is non-trivial due to their discrete decisions. We address this by adopting a differentiable relaxation that allows gradient-based end-to-end optimization. Extensive experiments on the LIBERO and SIMPLER benchmarks, as well as real-robot evaluations, show that our method achieves a 1.76× wall-clock inference speedup while simultaneously improving the average success rate by 1.9 percentage points (from 75.0% to 76.9%) on LIBERO and by 5.0 percentage points on real-world tasks, significantly outperforming existing baselines. This work highlights the potential of learning task-aware computational allocation policies, paving the way for VLA models that are both powerful and efficient.Code is available at GitHub.

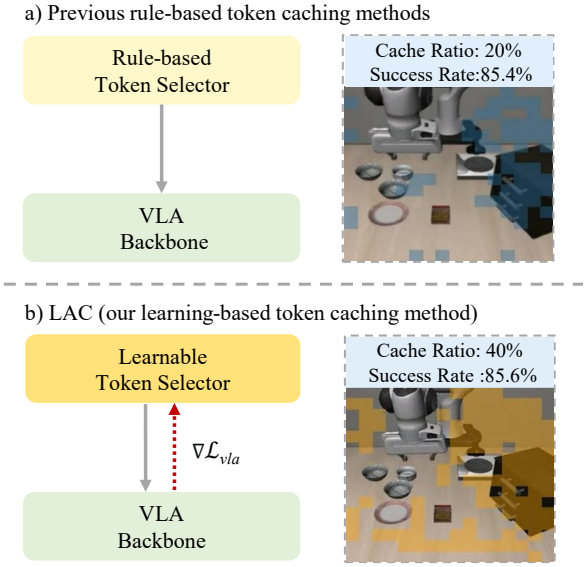

Figure 1. Our learnable caching (LAC) outperforms rule-based baselines in both efficiency and performance. **(a)** Rule-based selectors are decoupled from the task objective. They often rely on proxies, but *where the model attends is not necessarily what the task requires*. This leads to suboptimal, task-agnostic caching (20% cache ratio). **(b)** Our selector is optimized with direct task gradients ($\nabla\mathcal{L}_{\text{VLA}}$), ensuring its decisions align with actual task requirements. This enables a more aggressive adaptive caching strategy, leading to a much higher cache ratio (40%) and greater efficiency, while simultaneously improving the success rate.

1. Introduction

Large Vision-Language-Action (VLA) models have recently shown remarkable progress in unifying perception, reasoning, and control for embodied intelligence [4, 9, 15, 17, 20, 53, 55–57]. By coupling large-scale multimodal pretraining with autoregressive action decoding, models such as RT-2 [57], PaLM-E [9], and OpenVLA [17] enable robots to interpret open-ended language commands and perform complex manipulation tasks. However, deploying such models in real-world robotic systems remains challenging due to their high computational overhead, which conflicts with the low-latency requirements of real-time

* Equal Contribution
✉ Corresponding Author: shuoyang@hit.edu.cn

control, hindering widespread adoption.

A major inefficiency arises from redundant computation across consecutive frames, as VLA models reprocess the entire visual input at every step, even when most background regions remain static. Existing acceleration methods attempt to mitigate this by caching or pruning visual tokens, but they typically rely on fixed, rule-based heuristics [7, 48, 52]. As illustrated in Figure 1 (a), these rule-based selectors are fundamentally decoupled from the downstream task objective. This means their decisions are based on proxies, such as attention scores or visual saliency, which often fail to capture true task-relevance. A policy might waste computation on visually distracting but functionally irrelevant regions, or erroneously discard subtle but critical cues for the task. In essence, ***where the model attends is not necessarily what the task requires***. This fundamental misalignment leads to a suboptimal trade-off between speed and accuracy, limiting both the potential efficiency gains and the overall reliability of the system.

We argue that the key to efficient VLA inference lies not in manually designing better heuristics, but in learning a dynamic computation policy that is directly optimized for task success. We hypothesize that an optimal policy should learn to allocate computational resources based on scene motion and task requirements, reusing redundant information while recomputing only the most salient parts.

Based on this insight, we propose **LAC** (**L**earnable **A**daptive **C**aching for Vision-Language-Action models), a lightweight framework that transforms heuristic acceleration into a learnable, task-driven policy. As shown in Figure 1 (b), LAC introduces a *learnable token selector* that receives direct feedback from the final task loss ($\nabla \mathcal{L}_{vla}$) which measures the ability of VLA models. This end-to-end optimization enables the selector to make more intelligent, task-aware decisions. As a result, LAC can adopt a more aggressive caching strategy (40% cache ratio) that not only improves efficiency but also enhances task performance, achieving a higher success rate. At its core, LAC features two cooperative modules: the *Cached Token Selector*, which learns token-level dynamic saliency, and the *Cache Ratio Predictor*, which determines the overall reuse budget. To effectively assess scene dynamics, these modules leverage computationally lightweight optical flow. This approach provides a direct, pixel-level signal of motion, which is crucial for identifying which visual tokens correspond to moving objects like the robot's gripper or manipulated items. Crucially, we opt for this direct motion signal over language-based guidance, as the latter would likely necessitate a larger model and incur additional computational overhead, running counter to our goal of efficiency. Both are trained jointly through a task-driven policy learning framework, ensuring that every computation decision contributes directly to the final goal.

Before delving into the details, we summarize our contributions as follows:

- We propose **LAC**, the first learnable adaptive caching framework that transforms heuristic acceleration into an end-to-end trainable policy for VLA models.
- We design two lightweight and cooperative decision modules—the *Cached Token Selector*, which identifies which visual tokens should be recomputed or reused, and the *Cache Ratio Predictor*, which determines how many tokens to reuse based on scene-level motion entropy. Together, they enable fine-grained and adaptive token reuse for efficient VLA inference.
- Extensive experiments on LIBERO, SIMPLER, and real-world robotic platforms demonstrate that LAC achieves up to a 1.76× wall-clock acceleration while simultaneously improving task performance, outperforming existing rule-based pruning and caching baselines.

2. Related Work

Vision-Language-Action Models. Vision-Language-Action (VLA) models represent a significant advancement in robotics, building upon the multimodal reasoning capabilities of large-scale vision-language models (VLMs) [1, 2, 8, 10, 19, 28, 36, 41]. By introducing an action modality, these models [17, 20, 23, 25, 30, 31, 37, 38, 42, 43, 56, 57] are able to perform end-to-end visuomotor control. The common paradigm involves adapting powerful VLM backbones, such as Llama [11, 40], through fine-tuning on extensive robotics datasets like Open X-Embodiment [32]. To generate actions, these models employ diverse strategies, from discretizing robot movements into language-like tokens [17, 57] to integrating specialized decoders like diffusion action transformers [20, 44, 46] or flow models [4, 15]. This has led to successes in complex manipulation tasks [14, 54], but the substantial computational demands of VLA models remain a critical hurdle for practical, real-time deployment in resource-constrained settings.

Acceleration of VLA Models. Prior work on accelerating VLMs often falls short for robotic applications. Strategies like token pruning [5, 7, 26, 47, 52] and merging [6, 12, 24, 35] are ill-suited for robotics. They fail to exploit critical temporal redundancy across video frames. Moreover, their pruning logic can harm the spatial fidelity required for manipulation and introduces computational overhead that often outweighs the benefits for short robotic action sequences. VLA-specific approaches, including architectural optimizations [29, 45], dynamic scheduling [21, 49, 51], and quantization [33], demand costly retraining or rely on heuristic-based rules. High-frequency control methods [18, 34, 50] also leave the vision processing bottleneck largely unaddressed. The most related approach, VLA-Cache [48], introduces key-value caching across timesteps but employs a static, rule-based policy that cannot effec-

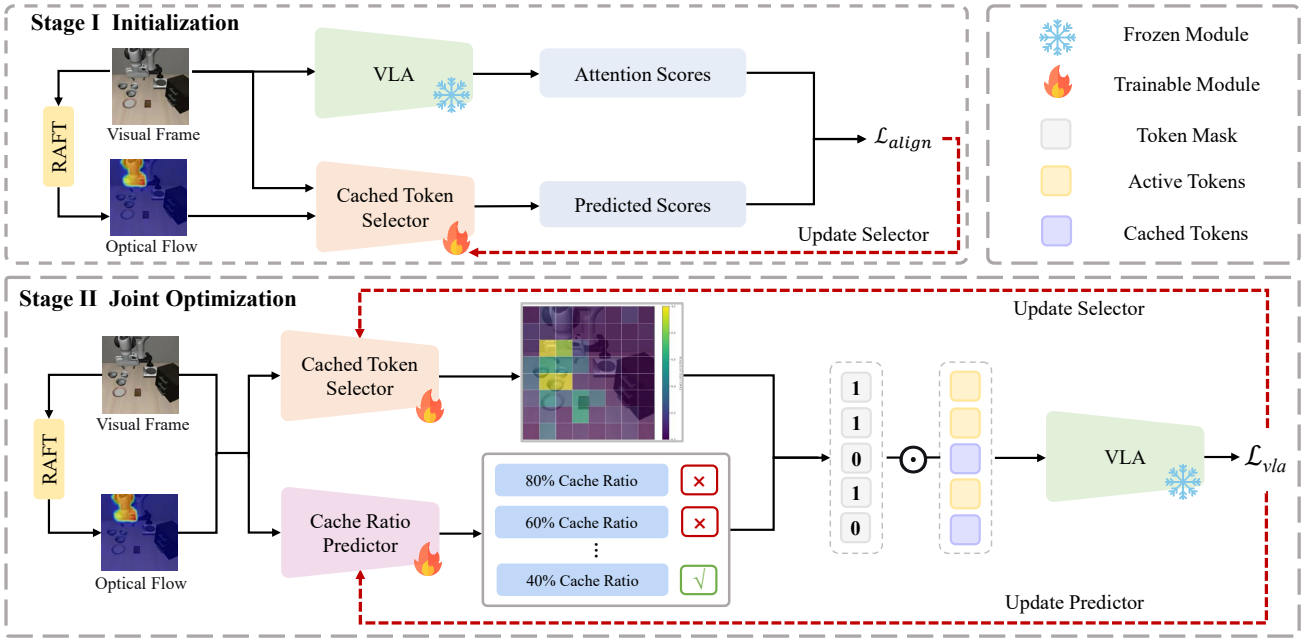

Figure 2. **Overview of the two-stage training framework for LAC. Stage I: Initialization.** The Cached Token Selector is initially pretrained to align with the VLA's internal attention map, providing a stable and informative initialization. **Stage II: Joint Optimization.** Following this, the Cached Token Selector and Cache Ratio Predictor are jointly optimized with the VLA backbone held frozen. The selector learns token-level saliency for dynamic reuse, while the predictor estimates the optimal cache ratio from a discrete set. Gradients from the task loss $\mathcal{L}_{\text{VLA}}$ are backpropagated through both modules via the differentiable selection mechanism, enabling end-to-end policy learning for adaptive computation.

tively adapt to changing scene dynamics.

In contrast, our proposed LAC transforms acceleration into a learnable, task-oriented policy, using jointly optimized modules to create a fine-grained, adaptive caching strategy that dynamically and effectively balances computation and accuracy.

3. Method

To address the high computational cost faced by Vision-Language-Action (VLA) models in real-time decision-making, we propose a dynamic computation optimization framework that enables adaptive efficiency within the perception–action loop. For embodied agents, efficient perception is the foundation of low-latency and responsive control. Our approach accelerates inference by intelligently reusing redundant visual information rather than performing uniform, dense computation for every incoming frame. The core of the method lies in two lightweight, cooperative modules that dynamically determine token-wise processing strategies based on scene content. The following sections detail the overall framework, key components, and the two-stage training procedure.

3.1. Overall Framework

To enhance the inference efficiency of VLA models, we propose a learnable adaptive caching framework that dynamically decides which visual tokens to recompute and which to reuse from a cache. This adaptive computation strategy aims to reduce computational overhead while simultaneously improving task performance. A high-level overview of our proposed method is illustrated in Figure 2.

At the heart of our framework are two lightweight, cooperative modules: the *Cached Token Selector* and the *Cache Ratio Predictor*. The Cached Token Selector is responsible for generating token-level saliency scores to identify which visual information is critical and requires recomputation. Concurrently, the Cache Ratio Predictor assesses the overall scene dynamics to determine an appropriate proportion of tokens to cache. These two modules operate as a plug-and-play addition, guiding the computational flow for a pre-trained and frozen VLA backbone, thereby preserving its powerful learned representations.

To train these decision-making modules effectively, we devise a two-stage training procedure. In the first stage, *Initialization*, we train the Cached Token Selector to mimic the attention distribution of the expert VLA model. This provides the selector with a strong initial policy for identifying salient regions. In the second stage, *Joint Optimization*, we

fine-tune the selector and train the Cache Ratio Predictor end-to-end. The optimization objective in this stage is the final task performance, ensuring that the learned caching policy is directly aligned with the ultimate goal.

3.2. Preliminaries: KV Caching for Efficiency

In autoregressive Transformer models, Key-Value (KV) caching is a standard technique to accelerate inference by reusing key ($\mathbf{K}$) and value ($\mathbf{V}$) representations across decoding steps. Given an input sequence $\mathbf{X} = [x_1, x_2, \ldots, x_T]$, the self-attention mechanism computes

$$\mathbf{Q} = \mathbf{X}W_Q, \quad \mathbf{K} = \mathbf{X}W_K, \quad \mathbf{V} = \mathbf{X}W_V, \tag{1}$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}. \tag{2}$$

During inference, only the new token's $\mathbf{k}_{\text{new}}$ and $\mathbf{v}_{\text{new}}$ are computed and appended to the cache as follows:

$$\mathbf{K}_t = [\mathbf{K}_{t-1}, \mathbf{k}_{\text{new}}], \quad \mathbf{V}_t = [\mathbf{V}_{t-1}, \mathbf{v}_{\text{new}}]. \tag{3}$$

This incremental update avoids recomputation of historical states, significantly improving decoding efficiency.

While KV caching works well for language decoding within a single context window, it remains limited for Vision-Language-Action (VLA) models. In robotic control, consecutive frames are highly redundant, yet current VLA models typically re-encode full visual inputs at every step, wasting computation and increasing latency.

This inefficiency motivates our work: most static regions need not be recomputed each timestep. By selectively updating only salient visual tokens and reusing cached representations for redundant ones, the model can achieve efficiency gains without sacrificing perception or control accuracy. Building on this intuition, our method learns an adaptive caching policy that dynamically decides *which* and *how many* to recompute, as detailed in the following sections.

3.3. Dynamic Token Caching Modules

At the core of our framework are two lightweight, cooperative modules: the *Cached Token Selector* and the *Cache Ratio Predictor*. To capture scene dynamics, they leverage a motion-aware representation $V_t = [I_t; O_t]$, formed by concatenating the visual frame I_t and the optical flow O_t. We compute O_t with a lightweight RAFT-small [39] model, as it provides a direct and efficient motion signal, avoiding the higher computational overhead associated with using language for guidance. These modules jointly decide *which* visual tokens to recompute and *how many* to reuse from the cache. For an input token sequence $X_t = \{x_t^{(1)}, \ldots, x_t^{(N)}\}$, they operate on V_t to produce a binary mask $M_t \in \{0, 1\}^N$ that guides the per-token computation.

3.3.1. Cached Token Selector

The Cached Token Selector is a function f_{sel} with parameters θ_{sel}, responsible for assessing token-level importance. Given the motion-aware input V_t, it generates a vector of saliency scores S_t:

$$S_t = f_{\text{sel}}(V_t; \theta_{\text{sel}}), \tag{4}$$

where $S_t = \{s_t^{(1)}, \ldots, s_t^{(N)}\}$ and $s_t^{(i)}$ is a continuous value in the range $[0, 1]$. A higher score $s_t^{(i)}$ indicates that token $x_t^{(i)}$'s information is critical for the current timestep—perhaps due to manipulator movement or object interaction—and should accordingly be recomputed. Conversely, tokens with lower scores are identified as candidates for caching. To ensure minimal computational overhead, f_{sel} is implemented as a small and efficient convolutional neural network (CNN).

3.3.2. Cache Ratio Predictor

While the selector provides a relative ranking, the Cache Ratio Predictor determines the overall computational budget. This module, $f_{\text{pred}}(\cdot; \theta_{\text{pred}})$, also assesses global scene dynamics using the input V_t to select an appropriate cache ratio from a discrete set $\mathcal{R} = \{r_1, \ldots, r_C\}$ by outputing a vector of logits L_t over the set $\mathcal{R}$:

$$L_t = f_{\text{pred}}(V_t; \theta_{\text{pred}}), \tag{5}$$

where $L_t = \{l_t^{(1)}, \ldots, l_t^{(C)}\}$. Each logit $l_t^{(j)}$ corresponds to the model's confidence in applying cache ratio r_j. During inference, the ratio with the highest logit is selected. In a static scene, the predictor is trained to select a high ratio to maximize efficiency, while in a dynamic scene, it learns to select a lower ratio to maintain high task performance.

3.4. Two-Stage Training Procedure

Learning a discrete caching policy from scratch is unstable under sparse supervision. We address this cold-start problem with a two-stage training paradigm: we first initialize the policy by distilling expert knowledge, then fine-tune it on task objectives while keeping the pretrained VLA frozen.

Stage I: Initialization via Attention Alignment. This stage provides the *Cached Token Selector* with an initialization by distilling knowledge from the expert VLA. Although the VLA's attention scores are not optimal policies, they offer a useful proxy for general visual saliency. As shown in Figure 2, the selector f_{sel} is trained to mimic the VLA's attention map S_{VLA} by minimizing a MSE loss:

$$\mathcal{L}_{\text{align}} = \text{MSE}(f_{\text{sel}}(V_t; \theta_{\text{sel}}), S_{\text{VLA}}). \tag{6}$$

This warm-up provides the selector with a sensible initial policy, which is crucial for stabilizing the subsequent end-to-end optimization process.

Stage II: Joint Optimization for Task Performance. After initialization, both modules are jointly optimized to balance task accuracy and computational efficiency. To make discrete selections differentiable, we employ the Gumbel-Softmax trick with straight-through estimation (see Sec. 3.5). The total loss combines the task loss $\mathcal{L}_{\text{VLA}}$ with a regularization term encouraging higher cache ratios:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{VLA}} + \lambda \mathcal{L}_{\text{ratio}}, \tag{7}$$

where

$$\mathcal{L}_{\text{ratio}} = -\mathbb{E}_{\tilde{p}_t}[r] = -\sum_{j=1}^{C} \tilde{p}_t^{(j)} r_j. \tag{8}$$

Here, $\tilde{p}_t$ denotes the soft probabilities from Gumbel-Softmax and r_j the candidate cache ratios. The weight λ controls the efficiency–performance trade-off. Gradients are backpropagated to update both modules. This end-to-end training aligns the learned caching policy directly with the downstream control objective.

3.5. Differentiable Discrete Decision Learning

A key challenge is that the decisions made by our modules are inherently discrete. The *Cache Ratio Predictor* uses an $\arg\max$ operation to select a ratio from a discrete set $\mathcal{R}$, while the *Cached Token Selector* relies on a non-differentiable `top-k` operation to identify tokens for caching. These operations obstruct gradient flow from the final task loss, preventing end-to-end optimization.

To overcome this obstacle, we introduce a differentiable policy optimization framework based on the Gumbel-Softmax trick [16] combined with the Straight-Through Estimator (STE) [3]. This creates a hybrid "hard forward, soft backward" strategy: it uses deterministic discrete selections in the forward pass for efficiency, while simultaneously enabling effective gradient flow through a continuous, "soft" approximation during backpropagation.

For the *Cache Ratio Predictor*, we use the Gumbel-Softmax trick. Given the module's output logits l_t, we then generate "soft" probability vector $\tilde{p}_t$ by adding Gumbel noise with a temperature τ:

$$\tilde{p}_t^{(j)} = \frac{\exp((l_t^{(j)} + g_j)/\tau)}{\sum_{k=1}^{C} \exp((l_t^{(k)} + g_k)/\tau)}, \tag{9}$$

where $g_j \sim \text{Gumbel}(0, 1)$. Then, in the forward pass, we perform a hard $\arg\max$ to get a one-hot vector p_t for deterministic ratio selection:

$$p_t = \text{one\_hot}(\arg\max_j(\tilde{p}_t^{(j)})). \tag{10}$$

During backpropagation, the Straight-Through Estimator (STE) passes gradients directly to the soft probabilities $\tilde{p}_t$.

For the *Cached Token Selector*, we apply a similar principle. While the forward pass uses a hard mask $\mathbf{M}_t$ from the 'top-k' operation, the backward pass constructs a differentiable "soft" mask $\tilde{\mathbf{M}}_t$ using a steep sigmoid function:

$$\tilde{M}_t^{(i)} = \sigma\left(\frac{s_t^{(i)} - \theta_k}{\tau_s}\right), \tag{11}$$

where θ_k is the score threshold for the k_t-th token and τ_s is a temperature parameter. Gradients are computed with respect to this soft mask. This hybrid strategy enables our decision modules to learn *what* and *how much* to cache in a unified, task-oriented framework.

3.6. Inference Procedure

Once the training process is complete, our decision modules are integrated with the frozen VLA backbone to create a highly efficient and stable inference pipeline.

Deterministic Decision-Making. For stable and consistent inference, we replace the stochastic sampling from training with deterministic $\arg\max$ operations. The *Cache Ratio Predictor* selects the cache ratio r_t corresponding to the highest logit. Subsequently, the *Cached Token Selector* generates a binary mask $\mathbf{M}_t$ by selecting the $k_t = N \cdot r_t$ tokens with the lowest importance scores.

Efficient Forward Pass. The generated mask $\mathbf{M}_t$ partitions visual tokens to enable an efficient forward pass. *Active tokens* are re-encoded to compute new Key (K) and Value (V) states, while the KV states of *cached tokens* are directly reused from the previous timestep. This merged set of KV states is then fed to the action decoder, significantly reducing redundant computation.

Stochastic Recovery Mechanism. To enhance long-horizon robustness and mitigate error accumulation from persistent caching, we employ a stochastic recovery mechanism. At each step, with a small probability p_{recover}, a fraction of "cached" tokens are randomly selected and forced to be recomputed. This low-cost refresh improves model stability without significantly impacting efficiency.

4. Experiments

To comprehensively assess the performance and efficiency of LAC, we conduct a rigorous evaluation across both diverse simulated environments and a physical robotic platform. Our framework is flexibly integrated with two prominent open-source Vision-Language-Action (VLA) models: OpenVLA [17] and CogAct [20]. These models are subsequently benchmarked on the LIBERO [27] and SIMPLER [22] environments, respectively.

4.1. Experimental Setup

Compared Methods. Given the shared architectural foundations of VLAs and Vision-Language Models (VLMs),

Table 1. **Results on the LIBERO benchmark.** Our method (LAC) simultaneously improves both performance and efficiency over all baselines. Compared to the strong OpenVLA baseline, LAC increases the average success rate by 1.9 percentage points (from 75.0% to 76.9%) while achieving a 1.76× wall-clock speedup and reducing FLOPs by 25.3%.

Method	LIBERO Spatial	LIBERO Object	LIBERO Goal	LIBERO Long	Average	FLOPs(T) ↓	CUDA Time (ms) ↓
Baseline (OpenVLA)	84.4%	**86.6%**	75.6%	53.2%	75.0%	1.864	51.91
SparseVLM [52]	79.8%	67.0%	72.6%	39.4%	64.7%	1.407	83.39
FastV [7]	83.4%	84.0%	74.2%	51.6%	73.3%	1.864	53.28
VLA-Cache [48]	83.8%	85.8%	76.4%	52.8%	74.7%	**1.355**	31.83
Ours (LAC)	**85.6%**	86.2%	**76.6%**	**59.2%**	**76.9%**	1.392	**29.51**

acceleration techniques developed for VLMs can often be adapted for VLA inference. Consequently, we apply two leading token-level acceleration techniques, SparseVLM [52] and FastV [7], to OpenVLA to serve as strong baseline comparisons on the LIBERO benchmark.

Evaluation Metrics. We quantify the performance of our method using three key metrics: success rate, FLOPs, and CUDA time. Success rate measures the efficacy of task completion. On the efficiency front, FLOPs represent the theoretical computational cost, whereas CUDA time provides a practical measure of wall-clock runtime on the GPU. This combination of metrics is standard in the evaluation of VLM/VLA acceleration frameworks.

4.2. Evaluation Benchmarks

LIBERO. The LIBERO benchmark [27] is structured into four specialized suites: Spatial, Object, Goal, and Long, each specifically designed to probe a distinct dimension of manipulation generalization. To maintain consistency and ensure reproducibility, we adopt the standard setup of OpenVLA [17], employing its officially provided model weights. Each suite consists of ten subtasks, and final performance is averaged over multiple trials.

SIMPLER. The SIMPLER simulator [22] features two primary configurations: Visual Matching and Variant Aggregation. In line with the methodology of CogAct [20], we evaluate our method on a Google robot arm across four distinct manipulation tasks within both settings. By using CogAct as our base model, we test the generalizability of LAC to VLA models with diverse action decoding heads and under varied simulation conditions.

Real-world Robot Setting We extend our evaluation to a physical Franka manipulator to assess performance under real-world conditions, including perception noise and actuation latency. For these experiments, we fine-tune the OpenVLA base model with LoRA [13]. The evaluation is conducted on four manipulation tasks: KnockCrisp, PickMango, CoverBanana, and KnockBottle.

4.3. Results on Simulation Environments

Main Results on LIBERO. Table 1 summarizes our main results on the LIBERO benchmark. Our method achieves a 1.76× wall-clock speedup and improves the average success rate by 1.9 percentage points. Notably, baselines such as SparseVLM show increased latency despite lower theoretical FLOPs. Their pruning logic introduces overhead that outweighs the benefits for the short action sequences in robotics, and can also harm the spatial fidelity required for manipulation tasks. These results confirm that LAC delivers substantial acceleration for VLA models while simultaneously enhancing task performance and generalization.

Qualitative Results on LIBERO. Figure 3 highlights the superiority of LAC's learned computational policy. LAC's saliency map (middle row) correctly identifies the moving gripper as the most critical region, ensuring it is always recomputed while the static background is efficiently cached (orange). Conversely, the rule-based method (top row) makes a critical error: its task-agnostic heuristic caches the target basket simply because it is initially static. This decision deprives the model of essential visual updates for the upcoming interaction, causing the policy to fail as the robot arm gets stuck at the basket's rim, unable to find the correct placement. LAC's policy, guided by the task objective, avoids this failure, and its stability is further enhanced by stochastic recovered tokens (green). The figure clearly illustrates that a learned policy is essential for selectively recomputing salient information, unlike rigid heuristics that can discard vital context.

Effect of Token Reusing Ratio. Figure 4 compares methods under different token reusing or pruning rates. As the reuse ratio increases, LAC maintains stable performance, whereas pruning-based methods (e.g., SparseVLM) exhibit steep drops in success rate. Moreover, our CUDA inference time remains consistently lower, indicating genuine wall-clock efficiency rather than theoretical FLOPs reduction.

Ablation Studies. We ablate LAC by progressively enabling its three core components (Table 2). Starting with the Token Selector alone achieves a 82.2% success rate, showing that learning token-level saliency already

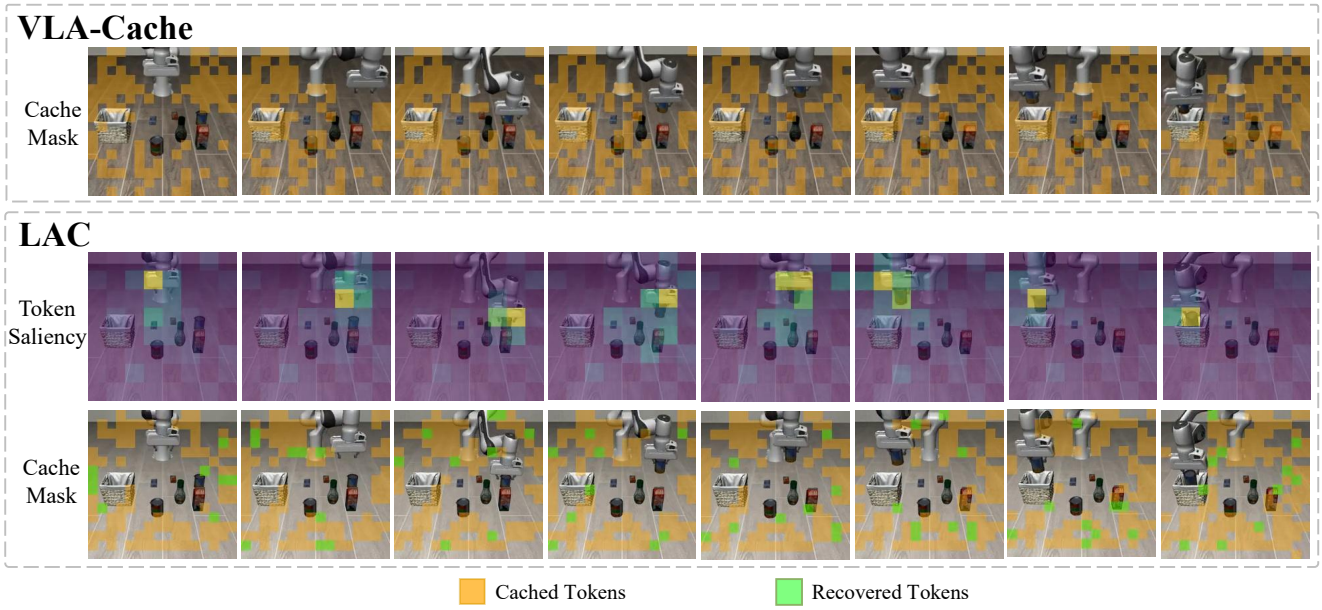

Figure 3. **Qualitative visualization of learned vs. rule-based caching.** LAC's policy (bottom) uses a learned saliency map to focus computation on the moving gripper while caching the static background (orange). In contrast, the rule-based method (top) detrimentally caches the static target basket, a task-agnostic error that could lead to task failure. This demonstrates the superiority of LAC's learned approach, which is further stabilized by recovered tokens (green).

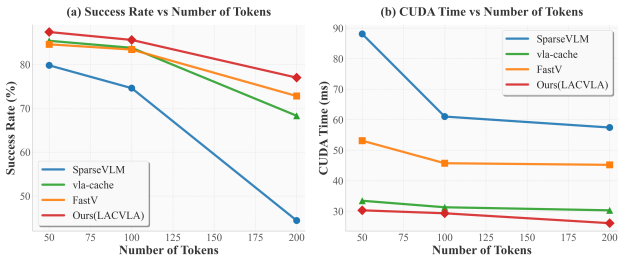

Figure 4. **Effect of Token Reusing/Pruning Ratio on LIBERO-Spatial.** Comparison of different methods under varying token reusing (or pruning) ratios. **Left:** Task success rate. **Right:** CUDA inference time. Our LAC maintains consistently high performance as the reuse ratio increases, while pruning-based methods such as SparseVLM degrade rapidly. In addition, LAC achieves the lowest CUDA latency across all settings, confirming its genuine wall-clock acceleration.

Table 2. Ablation on key components. Each row incrementally adds one module.

Method	Success (%)	FLOPs(T) ↓	Time (ms) ↓
Selector only	82.20	1.283	28.48
+ Reuse Predictor	83.40	1.325	29.04
+ Recovery (Full)	**85.60**	1.377	29.32

captures essential visuomotor cues for efficient reasoning. Adding the Reuse Predictor improves performance to 83.4%(+1.2points). The slight increase in FLOPs and time reflects its learned strategy: it adaptively reduces the cache ratio during critical moments to ensure task success. This demonstrates a successful trade-off, sacrificing a small amount of speed for a notable gain in robustness. This indicates that adaptive, scene-level reuse budgeting is effective. Finally, introducing the stochastic recovery mechanism yields the full model performance of 85.6% (+2.2 points over the previous variant), which enhances robustness and consistency across diverse task settings. These results verify that each component contributes progressively to efficiency–accuracy balance, forming a cohesive and complementary acceleration framework.

Main Results on SIMPLER. As shown in Table 3, our method achieves comparable or even better success rates than CogAct while significantly reducing computational cost. Specifically, our approach achieves an average success rate of 75.5% in the *Visual Matching* setting (CogAct: 74.8%) and 63.0% in the more challenging *Variant Aggregation* setting (CogAct: 61.3%). Furthermore, the efficiency improvements are substantial: our method reduces FLOPs by 17.4% (1.827 → 1.509) and achieves a 1.42× speedup in inference time (53.92 ms → 37.86 ms). These results highlight the strong portability of our approach, demonstrating its ability to serve as a general and architecture-agnostic acceleration strategy for VLA models with different action decoders (e.g., diffusion-based).

Table 3. Results on the SIMPLER benchmark [22] using CogAct [20] as the base model. Best results in each block are **bolded**.

SIMPLER	Method	PickCan	MoveNear	Drawer	DrawerApple	Average	FLOPs(T) ↓	CUDA Time (ms) ↓
Visual Matching	Baseline (CogAct)	91.3%	**85.0%**	71.8%	50.9%	74.8%	1.847	54.29
	VLA-Cache	92.0%	83.3%	70.5%	51.6%	74.4%	**1.496**	39.63
	Ours (LAC)	**92.3%**	84.2%	**72.7%**	**52.8%**	**75.5%**	1.511	**37.82**
Variant Aggregation	Baseline (CogAct)	89.6%	80.8%	28.3%	46.6%	61.3%	1.807	53.54
	VLA-Cache	91.7%	79.3%	**32.5%**	45.8%	62.3%	**1.493**	39.11
	Ours (LAC)	**92.1%**	**81.5%**	31.2%	**47.1%**	**63.0%**	1.506	**37.90**

Table 4. Real-world robotic manipulation results on four tasks.

Method	KnockCrisp	PickMango	CoverBanana	KnockBottle	Average	FLOPs(T) ↓	CUDA Time (ms) ↓
Baseline (OpenVLA)	48.0%	**16.0%**	**24.0%**	44.0%	33.0%	1.893	37.38
VLA-Cache	48.0%	12.0%	20.0%	40.0%	30.0%	**1.534**	33.36
Ours (LAC)	**52.0%**	12.0%	**24.0%**	**64.0%**	**38.0%**	1.569	**32.47**

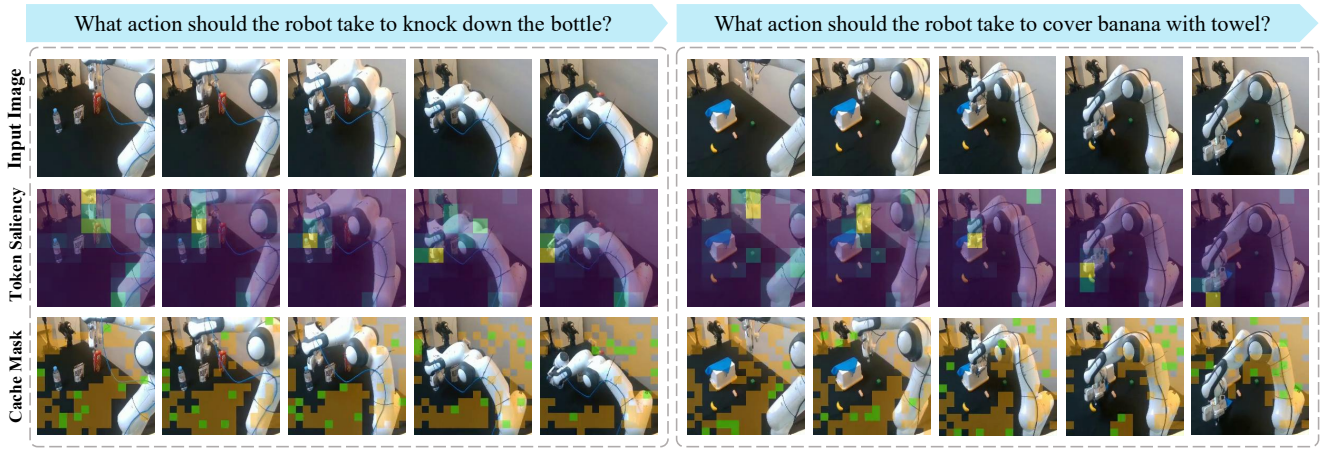

Figure 5. Qualitative visualization of our learned policy (LAC) on a real-world manipulation task. The Token Saliency map (middle row) demonstrates that the model correctly identifies the moving end-effector and its immediate interaction space as the most critical regions. Consequently, the Cache Mask (bottom row) shows an efficient policy where static background elements are cached, focusing computation only on task-relevant dynamics. Visualizations for two other tasks can be found in the Appendix.

4.4. Results on Real Robot

We validate our approach on a physical robot across four manipulation tasks, with quantitative results detailed in Table 4. Our method demonstrates superior performance, improving the average success rate by 5.0% over the baseline while reducing FLOPs and inference latency.

To provide qualitative insight into these results, Figure 5 visualizes the behavior of our learned policy. The Token Saliency map (middle row) confirms that our model learns to dynamically focus computation on critical, task-relevant regions—namely the robot's end-effector and its immediate interaction space. Consequently, as shown in the Cache Mask (bottom row), this learned saliency translates directly into an efficient caching strategy: static background elements are reused, while only the salient, moving regions are recomputed. This ability to intelligently ignore visual distractions not only explains the computational gains but also contributes to greater policy robustness and higher task success rates in cluttered, real-world environments.

5. Conclusion

This paper addresses the computational inefficiency of Vision-Language-Action (VLA) models for real-world robotics. We introduce a novel framework that transforms

inference acceleration from a heuristic-driven process into a learnable, task-oriented policy. At its core, our method, LAC, uses two lightweight, jointly optimized modules to dynamically decide which visual tokens to recompute and which to reuse from a cache.

Extensive experiments on simulation benchmarks and a physical robot demonstrate our approach's effectiveness. Our method, LAC, achieves up to a 1.76× inference speedup while improving task success rates, outperforming static caching and pruning methods. These results validate our core hypothesis: by learning to adaptively reuse visual information, a learnable, task-driven policy can enhance the efficiency, robustness, and overall performance of robotic control.

References

[1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. In *NeurIPS*, pages 23716–23736, 2022. 2

[2] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023. 2

[3] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013. 5

[4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Chelsea Finn, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 1, 2

[5] Jianjian Cao, Peng Ye, Shengze Li, Chong Yu, Yansong Tang, Jiwen Lu, and Tao Chen. MADTP: multimodal alignment-guided dynamic token pruning for accelerating vision-language transformer. In *CVPR*, pages 15710–15719. IEEE, 2024. 2

[6] Qingqing Cao, Bhargavi Paranjape, and Hannaneh Hajishirzi. Pumer: Pruning and merging tokens for efficient vision language models. In *ACL*, pages 12890–12903, 2023. 2

[7] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *ECCV*, pages 19–35. Springer, 2024. 2, 6

[8] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale N Fung, and Steven Hoi. Instructblip: Towards general-purpose vision-language models with instruction tuning. In *NeurIPS*, pages 49250–49267, 2023. 2

[9] Danny Driess, Fei Xia, Apoorva Anand, Aakanksha Chowdhery, Wenlong Huang, et al. Palm-e: An embodied multimodal language model. In *ICML*, pages 8469–8488. PMLR, 2023. 1

[10] Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, and Jie Tang. Glm: General language model pretraining with autoregressive blank infilling. In *ACL*, pages 320–335, 2022. 2

[11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. 2

[12] Anwen Hu, Haiyang Xu, Liang Zhang, Jiabo Ye, Ming Yan, Ji Zhang, Qin Jin, Fei Huang, and Jingren Zhou. mplug-docowl2: High-resolution compressing for ocr-free multi-page document understanding. In *ACL*, pages 5817–5834, 2025. 2

[13] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR*, page 3, 2022. 6

[14] Wenlong Huang, Chen Wang, Yunzhu Li, Ruohan Zhang, and Li Fei-Fei. Rekep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. In *CoRL*, pages 4573–4602. PMLR, 2024. 2

[15] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 2

[16] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. In *ICLR*, 2017. 5

[17] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. In *CoRL*, pages 2679–2713. PMLR, 2024. 1, 2, 5, 6

[18] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025. 2

[19] Junnan Li, Dongxu Li, Silvio Savarese, and Steven C. H. Hoi. BLIP-2: bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML*, pages 19730–19742. PMLR, 2023. 2

[20] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024. 1, 2, 5, 6, 8

[21] Wei Li, Renshan Zhang, Rui Shao, Jie He, and Liqiang Nie. Cogvla: Cognition-aligned vision-language-action model via instruction-driven routing & sparsification. *arXiv preprint arXiv:2508.21046*, 2025. 2

[22] Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, et al. Evaluating real-world robot manipulation policies in simulation. In *CoRL*, pages 3705–3728. PMLR, 2024. 5, 6, 8

[23] Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, et al. Vision-language foundation models as effective robot imitators. In *ICLR*. OpenReview.net, 2024. 2

[24] Yanwei Li, Chengyao Wang, and Jiaya Jia. Llama-vid: An image is worth 2 tokens in large language models. In *ECCV*, pages 323–340. Springer, 2024. 2

[25] Yi Li, Yuquan Deng, Jesse Zhang, Joel Jang, Marius Memmel, Caelan Reed Garrett, Fabio Ramos, Dieter Fox, Anqi Li, Abhishek Gupta, and Ankit Goyal. HAMSTER: hierarchical action models for open-world robot manipulation. In *ICLR*. OpenReview.net, 2025. 2

[26] Haokun Lin, Haoli Bai, Zhili Liu, Lu Hou, Muyi Sun, Linqi Song, Ying Wei, and Zhenan Sun. Mope-clip: Structured pruning for efficient vision-language models with module-wise pruning error metric. In *CVPR*, pages 27360–27370. IEEE, 2024. 2

[27] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS*, pages 44776–44791, 2023. 5, 6

[28] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. In *CVPR*, pages 26286–26296. IEEE, 2024. 2

[29] Jiaming Liu, Mengzhen Liu, Zhenyu Wang, Pengju An, et al. Robomamba: Efficient vision-language-action model for robotic reasoning and manipulation. In *NeurIPS*, pages 40085–40110, 2024. 2

[30] Jiaming Liu, Hao Chen, Pengju An, Zhuoyang Liu, Renrui Zhang, Chenyang Gu, Xiaoqi Li, Ziyu Guo, Sixiang Chen, Mengzhen Liu, et al. Hybridvla: Collaborative diffusion and autoregression in a unified vision-language-action model. *arXiv preprint arXiv:2503.10631*, 2025. 2

[31] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. RDT-1B: a diffusion foundation model for bimanual manipulation. In *ICLR*. OpenReview.net, 2025. 2

[32] Abby O'Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and RT-X models : Open x-embodiment collaboration. In *ICRA*, pages 6892–6903. IEEE, 2024. 2

[33] Seongmin Park, Hyungmin Kim, Wonseok Jeon, Juyoung Yang, Byeongwook Jeon, Yoonseon Oh, and Jungwook Choi. Quantization-aware imitation-learning for resource-efficient robotic control. *arXiv preprint arXiv:2412.01034*, 2024. 2

[34] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025. 2

[35] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. Llava-prumerge: Adaptive token reduction for efficient large multimodal models. In *ICCV*, pages 22857–22867, 2025. 2

[36] Leyang Shen, Gongwei Chen, Rui Shao, Weili Guan, and Liqiang Nie. Mome: Mixture of multimodal experts for generalist multimodal large language models. In *NeurIPS*, pages 42048–42070, 2024. 2

[37] Lucy Xiaoyang Shi, Brian Ichter, Michael Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. *arXiv preprint arXiv:2502.19417*, 2025. 2

[38] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. In *RSS*, 2024. 2

[39] Zachary Teed and Jia Deng. RAFT: recurrent all-pairs field transforms for optical flow. In *ECCV*, pages 402–419. Springer, 2020. 4

[40] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023. 2

[41] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 2

[42] Sicheng Wang, Sheng Liu, Weiheng Wang, Jianhua Shan, and Bin Fang. Robobert: An end-to-end multimodal robotic manipulation model. *arXiv preprint arXiv:2502.07837*, 2025. 2

[43] Junjie Wen, Jinqiang Cui, Zhenjun Zhao, Ruixin Yan, Zhi Gao, Lihua Dou, and Ben M. Chen. Syreanet: A physically guided underwater image enhancement framework integrating synthetic and real images. In *ICRA*, pages 5177–5183. IEEE, 2023. 2

[44] Junjie Wen, Yichen Zhu, Jinming Li, Zhibin Tang, Chaomin Shen, and Feifei Feng. Dexvla: Vision-language model with plug-in diffusion expert for general robot control. *arXiv preprint arXiv:2502.05855*, 2025. 2

[45] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE RA-L*, 2025. 2

[46] Junjie Wen, Yichen Zhu, Minjie Zhu, Zhibin Tang, Jinming Li, Zhongyi Zhou, Xiaoyu Liu, Chaomin Shen, Yaxin Peng, and Feifei Feng. Diffusionvla: Scaling robot foundation models via unified diffusion and autoregression. In *ICML*, 2025. 2

[47] Long Xing, Qidong Huang, Xiaoyi Dong, Jiajie Lu, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, Jiaqi Wang, Feng Wu, et al. Pyramiddrop: Accelerating your large vision-language models via pyramid visual redundancy reduction. *arXiv preprint arXiv:2410.17247*, 2024. 2

[48] Siyu Xu, Yunke Wang, Chenghao Xia, Dihao Zhu, et al. Vla-cache: Towards efficient vision-language-action model via adaptive token caching in robotic manipulation. *arXiv preprint arXiv:2502.02175*, 2025. 2, 6

[49] Yang Yue, Yulin Wang, Bingyi Kang, Yizeng Han, Shenzhi Wang, Shiji Song, Jiashi Feng, and Gao Huang. Deer-vla: Dynamic inference of multimodal large language models for efficient robot execution. In *NeurIPS*, pages 56619–56643, 2024. 2

[50] Jianke Zhang, Yanjiang Guo, Xiaoyu Chen, Yen-Jen Wang, Yucheng Hu, Chengming Shi, and Jianyu Chen. Hirt: Enhancing robotic control with hierarchical robot transformers. In *CoRL*, pages 933–946. PMLR, 2024. 2

[51] Rongyu Zhang, Menghang Dong, Yuan Zhang, Liang Heng, et al. Mole-vla: Dynamic layer-skipping vision language action model via mixture-of-layers for efficient robot manipulation. *arXiv preprint arXiv:2503.20384*, 2025. 2

[52] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, et al. Sparsevlm: Visual token sparsification for efficient vision-language model inference. *arXiv preprint arXiv:2410.04417*, 2024. 2, 6

[53] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, et al. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *CVPR*, pages 1702–1713, 2025. 1

[54] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *RSS*, 2023. 2

[55] Yifan Zhong, Xuchuan Huang, Ruochong Li, Ceyao Zhang, Zhang Chen, Tianrui Guan, Fanlian Zeng, Ka Num Lui, Yuyao Ye, Yitao Liang, et al. Dexgraspvla: A vision-language-action framework towards general dexterous grasping. *arXiv preprint arXiv:2502.20900*, 2025. 1

[56] Zhongyi Zhou, Yichen Zhu, Minjie Zhu, Junjie Wen, Ning Liu, Zhiyuan Xu, Weibin Meng, Yaxin Peng, Chaomin Shen, Feifei Feng, et al. Chatvla: Unified multimodal understanding and robot control with vision-language-action model. In *EMNLP*, pages 5377–5395, 2025. 2

[57] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. RT-2: vision-language-action models transfer web knowledge to robotic control. In *CoRL*, pages 2165–2183. PMLR, 2023. 1, 2

Appendix

A. Limitations

While LAC enables efficient adaptive inference, it presents two main limitations. First, our reliance on optical flow as a motion prior means that extreme visual conditions may affect the precision of the token selector. Second, in scenarios with rapid global visual changes or high-speed camera ego-motion, the policy will intelligently opt to recompute most tokens to preserve accuracy; consequently, the efficiency gains in such highly dynamic settings will naturally diminish compared to more stable environments.

B. Implementation Details

This section further details the technical implementation of our Learnable Adaptive Caching (LAC) framework within the Transformer's forward pass. At each inference step t, the binary mask $\mathbf{M}_t$ generated by our policy dictates the selective computation process. This is realized through the following modifications to the VLA decoder's forward pass:

- **Position Management and Attention Masking.** An internal state array tracks which tokens, as specified by $\mathbf{M}_t$, require recomputation. Tokens designated for caching preserve their positional encodings from the previous step. This is crucial as it allows the attention masks to be dynamically pruned, restricting the scope of the self-attention mechanism to the reduced set of active tokens and thus saving computation.
- **Conditional Rotary Embedding.** To introduce fresh positional information for the current timestep, rotary embeddings are applied exclusively to the recomputed tokens. Cached tokens bypass this operation, retaining their previously encoded states and avoiding redundant processing.
- **Dynamic Key-Value (KV) Cache Updates.** The KV cache at each Transformer layer l is updated dynamically. Newly computed tokens update their respective entries with $\{\mathbf{K}_t^l, \mathbf{V}_t^l\}$, while reused tokens inherit their values from the prior frame's cache, $\{\mathbf{K}_{t-1}^l, \mathbf{V}_{t-1}^l\}$. This partial update strategy, which mixes new and old KV entries, yields valid attention outputs. This is enabled by the permutation-invariant nature of the Transformer architecture, which allows attention to be computed over any valid set of key-value pairs.

This entire mechanism is fully compatible with the standard KV caching used for autoregressive decoding. The most significant computational gain is realized during the generation of the initial action token at each timestep; subsequent tokens are then decoded autoregressively without incurring additional cost.

C. Formal Efficiency Analysis

We provide a theoretical complexity analysis to quantify the computational benefits of LAC. Let N denote the number of visual tokens per frame, D the embedding dimension, L the number of Transformer layers, and M the intermediate dimension of the Feed-Forward Network (FFN).

Policy Overhead. The overhead of our method stems from the fixed-cost optical flow estimation (RAFT) and the lightweight CNN selector. This cost scales linearly with the input image resolution (H, W) and is independent of the Transformer depth or sequence length:

$$\mathcal{C}_{\text{policy}} \approx \mathcal{O}(H \cdot W \cdot C_{\text{cnn}}). \tag{12}$$

Since this overhead is computed only once per frame and does not scale with the model depth L, it remains negligible compared to the backbone computation.

Baseline Computational Cost. In a standard VLA Transformer layer, the computational cost consists of Multi-Head Self-Attention (MSA) and Feed-Forward Networks (FFN). For a sequence of N tokens, the FLOPs per layer are:

$$\mathcal{C}_{\text{base}} \approx \underbrace{4ND^2 + 2N^2D}_{\text{MSA}} + \underbrace{2NDM}_{\text{FFN}}. \tag{13}$$

Here, $4ND^2$ accounts for QKV and output projections, $2N^2D$ represents the attention score computation and aggregation, and $2NDM$ accounts for the two linear transformations in the FFN (projection $D \to M$ and $M \to D$).

LAC Computational Cost. Our method processes only a subset of active tokens $N_{\text{act}} = (1 - \rho)N$, where ρ is the cache ratio.

- **Projections & FFN:** We compute linear projections and FFNs strictly for active tokens.
- **Attention:** Active queries (N_{act}) attend to the full context (active + cached tokens, total N).

The reduced cost per layer is:

$$\mathcal{C}_{\text{lac}} \approx \underbrace{4N_{\text{act}}D^2 + 2(N_{\text{act}} \cdot N)D}_{\text{Partial MSA}} + \underbrace{2N_{\text{act}}DM}_{\text{Partial FFN}}. \tag{14}$$

Theoretical FLOPs Reduction. The reduction in FLOPs per layer is derived by subtracting the LAC cost from the baseline:

$$\Delta\text{FLOPs}_{\text{layer}} = \mathcal{C}_{\text{base}} - \mathcal{C}_{\text{lac}}. \tag{15}$$

Integrating the overhead, the total efficiency gain for the model is:

$$\Delta\text{FLOPs}_{\text{total}} \approx \sum_{l=1}^{L} \Delta\text{FLOPs}_{\text{layer}} - \mathcal{C}_{\text{policy}}. \tag{16}$$

This analysis demonstrates that LAC significantly reduces the theoretical computational complexity, at the cost of a minimal, constant policy overhead.

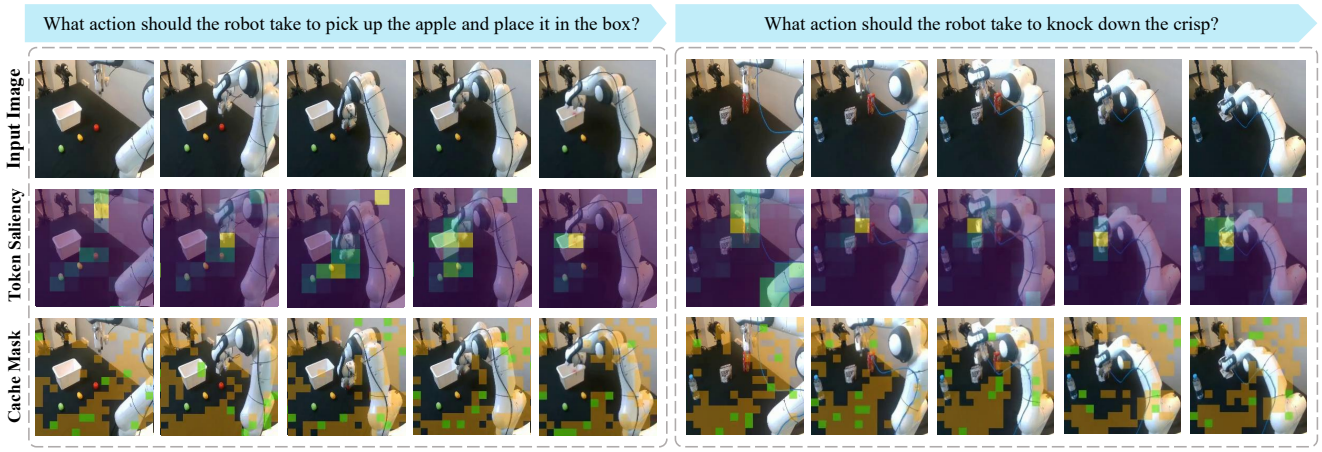

Figure 6. **Qualitative visualizations of the learned policy (LAC) on real-world tasks.** The examples show the policy correctly identifying task-relevant regions for recomputation. (Top) Input image sequence. (Middle) Learned Token Saliency map. (Bottom) Resulting Cache Mask, with orange for cached tokens and green for stochastically recovered tokens.

D. Additional Experiment Results

D.1. Real-World Robot Tasks

Figure 6 visualizes our policy's behavior on two real-world tasks: *"pick up the apple and place it in the box"* (left) and *"knock down the crisp"* (right). The learned Token Saliency (middle row) demonstrates the model's ability to effectively distinguish between important and unimportant visual regions. High saliency scores are concentrated on the robot's end-effector and the interaction targets, while the static background is assigned low importance. Consequently, the Cache Mask (bottom row) reveals an efficient strategy: unimportant visual regions are cached (marked in orange), while the critical, high-saliency regions remain unmasked for active recomputation. Additionally, green tiles appear within the cached regions, illustrating the stochastic recovery mechanism that randomly updates a fraction of tokens to ensure robustness.

D.2. LIBERO Benchmark

Figures 7 and 8 present additional qualitative visualizations across four diverse tasks from the LIBERO benchmark. Consistent with our real-world observations, the learned policy accurately identifies task-relevant dynamics: high saliency scores are assigned to the robot gripper and target objects, while the static environment is cached for reuse. The resulting Cache Masks in both figures confirm that LAC efficiently allocates computation by re-computing critical regions (unmasked) and retrieving the static background from the cache (orange), with occasional updates provided by stochastic recovery (green) to prevent error accumulation.

D.3. Additional Ablations and Comparisons

To further justify our architectural decisions and training paradigm, we conduct additional ablation studies on the LIBERO benchmark. Specifically, we investigate the necessity of the Two-Stage Training strategy and the choice of input modality for the decision modules. The quantitative results are summarized in Table 5.

Method Variant	**Success Rate (%)**
w/o Stage I Initialization	79.2
Language-guided Policy	83.0
Ours (LAC)	**85.6**

Table 5. **Additional Ablation Studies on LIBERO.** We compare our full method against a variant without Stage I initialization and a variant replacing optical flow with language guidance.

Impact of Initialization Strategy (Stage I). Our framework employs a two-stage training pipeline where the *Cached Token Selector* is first initialized by distilling attention scores from the frozen VLA (Stage I) before end-to-end optimization. As shown in Table 5, skipping this initialization step ("w/o Stage I Initialization") results in a performance drop to 79.2%. This confirms that learning a discrete selection policy from scratch under sparse supervision is unstable. The initialization phase provides a critical "visual saliency prior," ensuring the selector begins the joint optimization (Stage II) from a reasonable state rather than converging to suboptimal local minima.

Motion vs. Language for Adaptive Caching. We also evaluated an alternative design where the LAC framework is conditioned on language instructions rather than optical

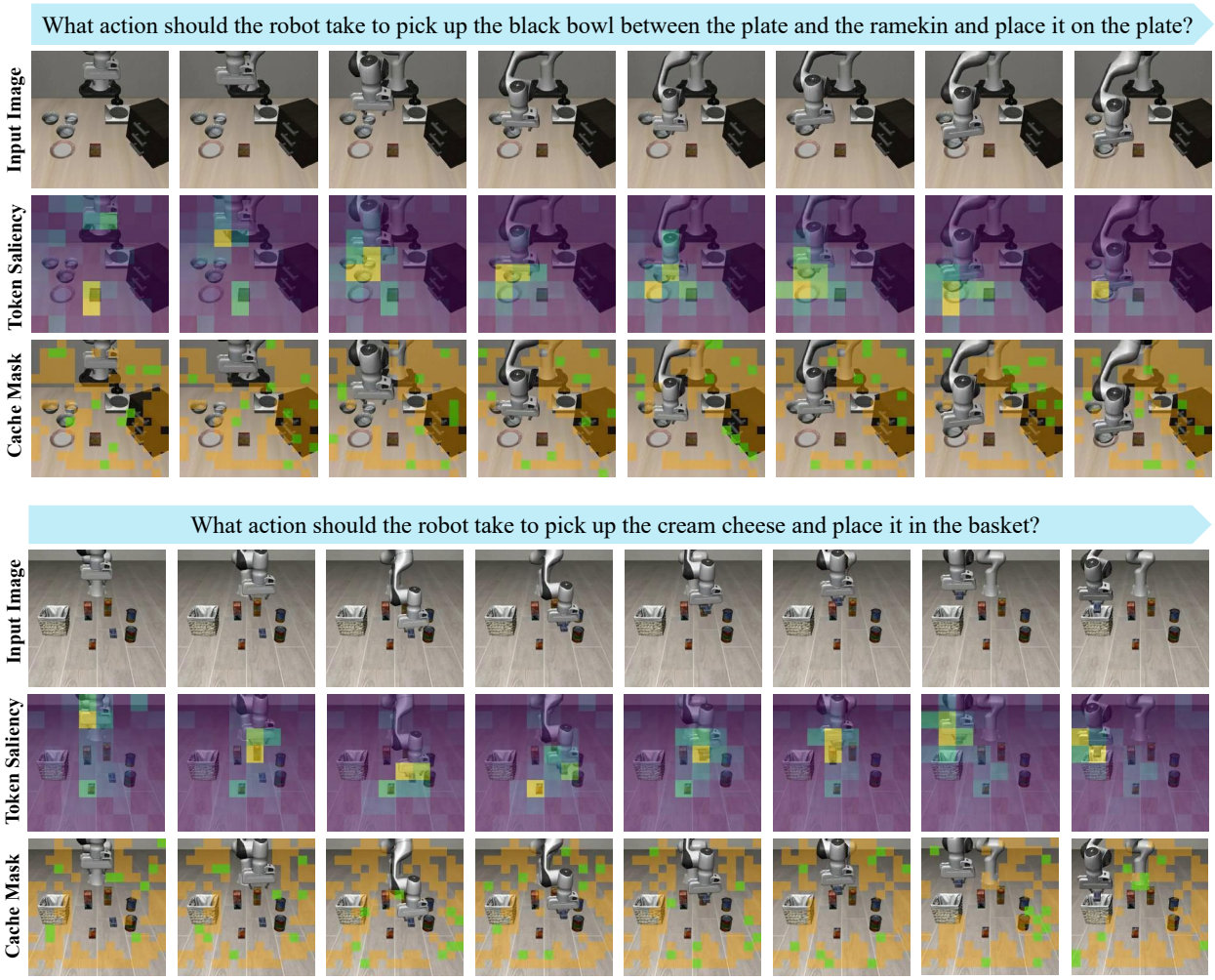

Figure 7. **Qualitative Results on LIBERO Benchmark (Part 1).** We visualize the behavior of LAC on the first set of manipulation tasks. (Top) Input image sequence; (Middle) Learned Token Saliency map; (Bottom) Generated Cache Mask (orange for cached, green for stochastically recovered, and unmasked regions for active). High saliency is consistently assigned to task-relevant regions like the gripper and target object, which are actively recomputed.

flow. This variant achieves a success rate of 83.0%, underperforming our motion-based approach. Consistent with the discussion in our main text, relying on language guidance presents a dilemma between accuracy and efficiency. Effective vision-language alignment typically requires large, computationally expensive encoders. In this ablation, to maintain an inference speed comparable to LAC, we employed a lightweight language conditioning module. However, this restricted capacity limits the model's ability to precisely align textual instructions with visual features. In contrast, optical flow provides a direct, pixel-level signal of motion that naturally captures task-relevant dynamics without the heavy computational overhead required for deep semantic processing.

Figure 8. **Qualitative Results on LIBERO Benchmark (Part 2).** Visualization on additional tasks. Similar to Part 1, the policy successfully caches the unimportant visual regions (orange) while actively re-computing the moving agents and interaction targets (shown as unmasked). Green tiles indicate tokens updated via the stochastic recovery mechanism, demonstrating the robustness of our adaptive caching strategy.